

LAPORAN PRAKTIKUM

STRUKTUR DATA

BINARY TREE

PEKAN 9



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Binary Tree menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Dasar Teori.....	3
2.2 Kode Program.....	9
2.3 Langkah Kerja.....	13
2.4 Analisis Hasil.....	28
BAB 3 KESIMPULAN	
3.1 Hasil Praktikum.....	29
3.2 Saran Pengembangan.....	32
DAFTAR KEPUSTAKAAN.....	33
TUGAS PRAKTIKUM PEKAN 9.....	34

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu materi penting dalam pemrograman karena digunakan untuk mengatur dan menyimpan data agar dapat diakses serta dikelola dengan lebih efisien. Salah satu struktur data yang sering digunakan adalah Binary Tree, yaitu struktur data berbentuk hierarki yang terdiri dari root, parent, dan child node. Berbeda dengan struktur data linear seperti array atau linked list, Binary Tree memungkinkan data disusun dalam bentuk percabangan sehingga hubungan antar data dapat direpresentasikan dengan lebih jelas.

Pada praktikum ini dilakukan pembuatan Binary Tree menggunakan bahasa pemrograman Java dengan menambahkan beberapa node dan menghubungkannya menjadi sebuah pohon biner. Selain membangun struktur pohon, praktikum juga mencakup proses perhitungan jumlah simpul, pencarian data, serta penelusuran pohon menggunakan metode traversal seperti InOrder, PreOrder, dan PostOrder. Melalui proses tersebut, mahasiswa dapat melihat bagaimana setiap node saling terhubung dan bagaimana data diakses berdasarkan aturan traversal yang berbeda.

Pemahaman mengenai Binary Tree menjadi penting karena konsep ini menjadi dasar bagi berbagai struktur data dan algoritma yang lebih kompleks, seperti Binary Search Tree, Heap, dan Tree Traversal pada sistem basis data maupun sistem operasi. Dengan melakukan praktikum secara langsung, mahasiswa tidak hanya memahami konsep Binary Tree secara teori, tetapi juga mampu mengimplementasikan dan menganalisis cara kerja struktur data tersebut dalam sebuah program.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar Binary Tree beserta hubungan antara root, parent, child, dan leaf node dalam struktur pohon biner.
2. Mengimplementasikan Binary Tree menggunakan bahasa pemrograman Java dengan menambahkan node serta membentuk struktur pohon sesuai rancangan yang telah ditentukan.
3. Menganalisis hasil traversal Binary Tree menggunakan metode InOrder, PreOrder, dan PostOrder serta memahami perbedaan urutan kunjungan node pada masing-masing metode.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Menambah pemahaman mahasiswa mengenai penerapan struktur data Binary Tree dalam pemrograman dan pengolahan data.
2. Melatih kemampuan mahasiswa dalam mengimplementasikan konsep Binary Tree ke dalam bentuk program menggunakan bahasa Java.
3. Membantu mahasiswa memahami proses traversal dan pengelolaan node pada Binary Tree sebagai dasar untuk mempelajari struktur data yang lebih kompleks di masa mendatang.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Binary Tree merupakan salah satu struktur data non-linear yang menyimpan data dalam bentuk hierarki atau percabangan. Pada struktur ini, setiap node dapat memiliki maksimal dua anak, yaitu anak kiri (*left child*) dan anak kanan (*right child*). Node yang berada pada posisi paling atas disebut *root*, sedangkan node yang tidak memiliki anak disebut *leaf node*. Struktur Binary Tree banyak digunakan dalam berbagai aplikasi komputer karena mampu merepresentasikan hubungan data secara terstruktur dan memudahkan proses pencarian maupun penelusuran data.

Dalam implementasinya, sebuah Binary Tree tersusun dari kumpulan node yang saling terhubung. Setiap node umumnya menyimpan data serta referensi menuju anak kiri dan anak kanan. Hubungan antar node membentuk sebuah pohon yang memungkinkan data diakses melalui berbagai jalur traversal. Pada praktikum ini, Binary Tree dibangun dengan membuat node-node secara manual, kemudian menghubungkannya sehingga membentuk struktur pohon yang dapat ditelusuri menggunakan beberapa metode traversal.

➤ **Komponen Utama Binary Tree**

Beberapa komponen penting yang terdapat pada Binary Tree antara lain:

1. Root

Node yang berada pada posisi paling atas dan menjadi titik awal dari seluruh struktur pohon.

2. Parent Node

Node yang memiliki satu atau lebih anak (*child node*).

3. Child Node

Node yang terhubung langsung di bawah parent node.

4. Leaf Node

Node yang tidak memiliki anak kiri maupun anak kanan.

5. Subtree

Bagian dari Binary Tree yang juga membentuk struktur pohon tersendiri di bawah suatu node.

➤ Traversal pada Binary Tree

Traversal adalah proses mengunjungi seluruh node yang ada pada pohon dengan urutan tertentu. Traversal digunakan untuk membaca, menampilkan, atau memproses data yang tersimpan pada Binary Tree. Pada praktikum ini digunakan tiga metode traversal, yaitu InOrder, PreOrder, dan PostOrder.

1. InOrder Traversal

InOrder Traversal dilakukan dengan urutan:

1. Mengunjungi subtree kiri.
2. Mengunjungi root.
3. Mengunjungi subtree kanan.

Metode ini sering digunakan pada Binary Search Tree karena dapat menghasilkan data yang terurut dari nilai terkecil ke terbesar. Pada praktikum, traversal InOrder menghasilkan urutan kunjungan node yang berbeda dibandingkan metode traversal lainnya karena root tidak dikunjungi terlebih dahulu.

2. PreOrder Traversal

PreOrder Traversal dilakukan dengan urutan:

1. Mengunjungi root.
2. Mengunjungi subtree kiri.
3. Mengunjungi subtree kanan.

Metode ini digunakan ketika struktur pohon ingin diproses mulai dari akar terlebih dahulu. Pada hasil praktikum, node root muncul pada urutan pertama karena proses traversal selalu dimulai dari akar sebelum berpindah ke cabang lainnya.

3. PostOrder Traversal

PostOrder Traversal dilakukan dengan urutan:

1. Mengunjungi subtree kiri.
2. Mengunjungi subtree kanan.
3. Mengunjungi root.

Metode ini biasanya digunakan ketika seluruh anak dari suatu node harus diproses terlebih dahulu sebelum node induknya. Pada hasil praktikum, root muncul pada urutan terakhir karena proses traversal baru mengunjungi akar setelah seluruh cabang selesai ditelusuri.

➤ Perhitungan Jumlah Simpul (Node Counting)

Selain traversal, Binary Tree juga dapat digunakan untuk menghitung jumlah simpul yang terdapat pada pohon. Perhitungan dilakukan secara rekursif dengan menjumlahkan node saat ini, kemudian menambahkan jumlah node pada subtree kiri dan subtree kanan. Jika suatu node bernilai null, maka nilai yang dikembalikan adalah 0. Teknik ini memungkinkan seluruh node pada pohon dihitung secara otomatis tanpa harus memeriksa satu per satu.

➤ **Rekursi pada Binary Tree**

Rekursi merupakan teknik pemrograman yang banyak digunakan pada struktur Binary Tree karena bentuk pohon yang bercabang secara alami sesuai dengan konsep pemanggilan fungsi berulang. Pada praktikum ini, rekursi digunakan dalam proses traversal dan perhitungan jumlah simpul. Setiap fungsi akan memanggil dirinya sendiri untuk mengakses subtree kiri dan subtree kanan hingga mencapai node terakhir. Dengan pendekatan ini, kode program menjadi lebih sederhana dan mudah dipahami dibandingkan menggunakan banyak perulangan.

➤ **Breadth First Search (BFS)**

Breadth First Search (BFS) merupakan algoritma penelusuran graf yang bekerja dengan cara mengunjungi node berdasarkan tingkat kedekatannya dari node awal. Proses pencarian dilakukan secara melebar (level by level), sehingga semua node yang berada pada level yang sama akan dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya. Dalam implementasinya, BFS menggunakan struktur data **Queue (antrian)** yang menerapkan prinsip First In First Out (FIFO). Karena menelusuri graf secara bertahap berdasarkan level, BFS sering digunakan untuk mencari jalur terpendek pada graf yang tidak memiliki bobot.

Karakteristik BFS:

- Menggunakan struktur data Queue.
- Menelusuri node per level atau secara melebar.
- Cocok untuk mencari jalur terpendek pada graf tanpa bobot.
- Setiap node dikunjungi satu kali untuk menghindari perulangan.
- Membutuhkan memori lebih besar karena menyimpan banyak node dalam antrian.

➤ **Depth First Search (DFS)**

Depth First Search (DFS) merupakan algoritma penelusuran graf yang bekerja dengan cara menelusuri satu cabang hingga mencapai node terdalam terlebih dahulu sebelum kembali ke node sebelumnya untuk mencari cabang lain. Proses ini dikenal dengan istilah backtracking. DFS dapat diimplementasikan menggunakan struktur data **Stack (tumpukan)** atau menggunakan rekursi. Berbeda dengan BFS, DFS tidak selalu menghasilkan jalur terpendek, tetapi algoritma ini lebih sederhana dan efisien dalam penggunaan memori pada beberapa kasus.

Karakteristik DFS:

- Menggunakan struktur data Stack atau rekursi.
- Menelusuri node hingga ke kedalaman maksimum terlebih dahulu.
- Menggunakan proses backtracking untuk kembali ke cabang sebelumnya.
- Tidak selalu menghasilkan jalur terpendek.
- Umumnya membutuhkan memori lebih sedikit dibandingkan BFS.

2.2 Kode Program

2.2.1 Node_2511531016

```
package pekan9_2511531016;

public class Node_2511531016 {
    int data_1016;
    Node_2511531016 left_1016;
    Node_2511531016 right_1016;

    public Node_2511531016(int data_1016) {
        this.data_1016 = data_1016;
        left_1016 = null;
        right_1016 = null;
    }
    public void setLeft_1016(Node_2511531016 node_1016) {
        if (left_1016 == null)
            left_1016 = node_1016;
    }
    public void setRight_1016(Node_2511531016 node_1016) {
        if (right_1016 == null)
            right_1016 = node_1016;
    }
    public Node_2511531016 getLeft_1016() {
        return left_1016;
    }
    public Node_2511531016 getRight_1016() {
        return right_1016;
    }
    public int getData_1016() {
        return data_1016;
    }
    public void setData_1016(int data_1016) {
        this.data_1016 = data_1016;
    }

    void printPreorder_1016(Node_2511531016 node_1016) {
        if (node_1016 == null)
            return;
        System.out.print(node_1016.data_1016 + " ");
        printPreorder_1016(node_1016.left_1016);
        printPreorder_1016(node_1016.right_1016);
    }
    void printPostorder_1016(Node_2511531016 node_1016) {
        if (node_1016 == null)
            return;
        printPostorder_1016(node_1016.left_1016);
        printPostorder_1016(node_1016.right_1016);
        System.out.print(node_1016.data_1016 + " ");
    }
    void printInorder_1016(Node_2511531016 node_1016) {
        if (node_1016 == null)
            return;
        printInorder_1016(node_1016.left_1016);
        System.out.print(node_1016.data_1016 + " ");
        printInorder_1016(node_1016.right_1016);
    }
    public String print() {
        return this.print("", true, "");
    }
    public String print(String prefix_1016, boolean isTail_1016, String sb_1016) {
        if (right_1016 != null) {
            right_1016.print(prefix_1016 + (isTail_1016 ? "| " : " "), false, sb_1016);
        }
        System.out.println(prefix_1016 + (isTail_1016 ? "\\-- " : "/--") + data_1016);
        if (left_1016 != null) {
            left_1016.print(prefix_1016 + (isTail_1016 ? " " : "| "), true, sb_1016);
        }
        return sb_1016;
    }
}
```

Kode Program 2.1 (Kode Program Node)

2.2.2 BTree_2511531016

```
package pekan9_2511531016;

public class BTree_2511531016 {
    private Node_2511531016 root_1016;
    private Node_2511531016 currentNode_1016;
    public BTree_2511531016() {
        root_1016 = null;
    }
    public boolean search_1016(int data_1016) {
        return search_1016(root_1016, data_1016);
    }
    private boolean search_1016(Node_2511531016 node_1016, int data_1016) {
        if (node_1016.getData_1016() == data_1016)
            return true;
        if (node_1016.getLeft_1016() != null)
            if (search_1016(node_1016.getLeft_1016(), data_1016))
                return true;
        if (node_1016.getRight_1016() != null)
            if (search_1016(node_1016.getRight_1016(), data_1016))
                return true;
        return false;
    }
    public void printInorder_1016() {
        root_1016.printInorder_1016(root_1016);
    }
    public void printPreorder_1016() {
        root_1016.printPreorder_1016(root_1016);
    }
    public void printPostorder_1016() {
        root_1016.printPostorder_1016(root_1016);
    }
    public Node_2511531016 getRoot_1016() {
        return root_1016;
    }
    public boolean isEmpty() {
        return root_1016 == null;
    }
    public int countNodes_1016() {
        return countNodes_1016(root_1016);
    }
    private int countNodes_1016(Node_2511531016 node_1016) {
        int count_1016 = 1;
        if (node_1016 == null) {
            return 0;
        } else {
            count_1016 += countNodes_1016(node_1016.getLeft_1016());
            count_1016 += countNodes_1016(node_1016.getRight_1016());
            return count_1016;
        }
    }
    public void print_1016() {
        root_1016.print();
    }
    public Node_2511531016 getCurrent_1016() {
        return currentNode_1016;
    }
    public void setCurrent_1016(Node_2511531016 node_1016) {
        this.currentNode_1016 = node_1016;
    }
    public void setRoot_1016(Node_2511531016 root_1016) {
        this.root_1016 = root_1016;
    }
}
```

Kode Program 2.2 (Kode Program BTree)

2.2.3 BTreeDriver_2511531016

```
package pekan9_2511531016;

public class BTreeDriver_2511531016 {
    public static void main(String[] args) {
        // Membuat pohon
        BTree_2511531016 tree_1016 = new BTree_2511531016();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_1016.countNodes_1016());
        // Menambahkan simpul data 1
        Node_2511531016 root_1016 = new Node_2511531016(1);
        // Menjadikan simpul 1 sebagai root
        tree_1016.setRoot_1016(root_1016);
        System.out.println("Jumlah simpul jika hanya ada root: ");
        System.out.println(tree_1016.countNodes_1016());

        Node_2511531016 node2 = new Node_2511531016(2);
        Node_2511531016 node3 = new Node_2511531016(3);
        Node_2511531016 node4 = new Node_2511531016(4);
        Node_2511531016 node5 = new Node_2511531016(5);
        Node_2511531016 node6 = new Node_2511531016(6);
        Node_2511531016 node7 = new Node_2511531016(7);
        Node_2511531016 node8 = new Node_2511531016(8);
        Node_2511531016 node9 = new Node_2511531016(9);

        root_1016.setLeft_1016(node2);
        node2.setLeft_1016(node4);
        node2.setRight_1016(node5);
        node4.setRight_1016(node8);
        root_1016.setRight_1016(node3);
        node3.setLeft_1016(node6);
        node3.setRight_1016(node7);
        node6.setLeft_1016(node9);
        // Set root
        tree_1016.setCurrent_1016(tree_1016.getRoot_1016());
        System.out.println("Menampilkan simpul terakhir: ");
        System.out.println(tree_1016.getCurrent_1016().getData_1016());
        System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
        System.out.println(tree_1016.countNodes_1016());
        System.out.println("InOrder: ");
        tree_1016.printInorder_1016();
        System.out.println("\nPreorder: ");
        tree_1016.printPreorder_1016();
        System.out.println("\nPostorder: ");
        tree_1016.printPostorder_1016();
        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_1016.print_1016();
    }
}
```

Kode Program 2.3 (Kode Program BTreeDriver)

2.2.4 GraphTraversal_2511531016

```
package pekan9_2511531016;
import java.util.*;

public class GraphTraversal_2511531016 {
    private Map<String, List<String>> graph_1016 = new HashMap<>();
    // Menambahkan edge (graf tak berarah)
    public void addEdge(String node1, String node2) {
        graph_1016.putIfAbsent(node1, new ArrayList<>());
        graph_1016.putIfAbsent(node2, new ArrayList<>());
        graph_1016.get(node1).add(node2);
        graph_1016.get(node2).add(node1);
    }

    // Menampilkan graf awal
    public void printGraph() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_1016 : graph_1016.keySet()) {
            System.out.print(node_1016 + " -> ");
            List<String> neighbors_1016 = graph_1016.get(node_1016);
            System.out.println(String.join(", ", neighbors_1016));
        }
        System.out.println();
    }

    // DFS rekursif
    public void dfs(String start_1016) {
        Set<String> visited_1016 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper(start_1016, visited_1016);
        System.out.println();
    }

    private void dfsHelper(String current_1016, Set<String> visited_1016) {
        if (visited_1016.contains(current_1016)) return;
        visited_1016.add(current_1016);
        System.out.print(current_1016 + " ");
        for (String neighbor_1016 : graph_1016.getDefault(current_1016, new ArrayList<>())) {
            dfsHelper(neighbor_1016, visited_1016);
        }
    }

    // BFS iteratif
    public void bfs(String start_1016) {
        Set<String> visited_1016 = new HashSet<>();
        Queue<String> queue_1016 = new LinkedList<>();
        queue_1016.add(start_1016);
        visited_1016.add(start_1016);
        System.out.println("Penelusuran BFS:");
        while (!queue_1016.isEmpty()) {
            String current_1016 = queue_1016.poll();
            System.out.print(current_1016 + " ");

            for (String neighbor_1016 : graph_1016.getDefault(current_1016, new ArrayList<>())) {
                if (!visited_1016.contains(neighbor_1016)) {
                    queue_1016.add(neighbor_1016);
                    visited_1016.add(neighbor_1016);
                }
            }
        }
        System.out.println();
    }

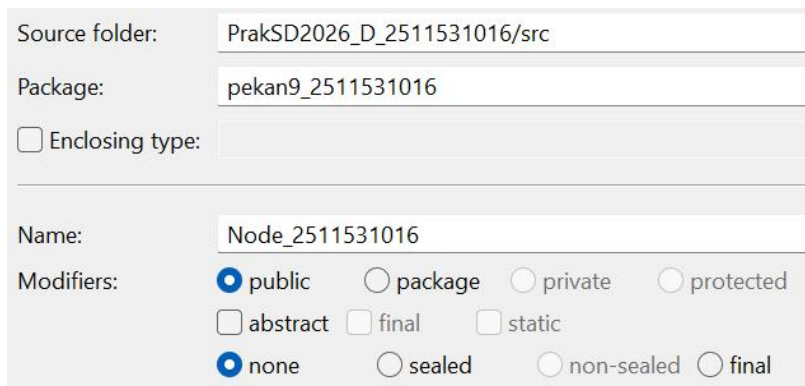
    // Main
    public static void main(String[] args) {
        GraphTraversal_2511531016 graph_1016 = new GraphTraversal_2511531016();
        // Contoh graf: A-B, A-C, B-D, B-E
        graph_1016.addEdge("A", "B");
        graph_1016.addEdge("A", "C");
        graph_1016.addEdge("B", "D");
        graph_1016.addEdge("B", "E");
        // Cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_1016.printGraph();
        // Lakukan penelusuran
        graph_1016.dfs("A");
        graph_1016.bfs("A");
    }
}
```

Kode Program 2.4 (Kode Program GraphTraversal)

2.3 Langkah Kerja

2.3.1 Node_2511531016

1. Membuat kelas baru bernama *Node_2511531016* dalam *package pekan9_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan9_2511531016
☐ Enclosing type:	
Name:	Node_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.1 (Langkah 1 Node)

2. Program diawali dengan deklarasi package dan pembuatan class. Class ini berfungsi sebagai representasi sebuah node pada struktur data Binary Tree. Di dalam class dideklarasikan atribut `data_1016` untuk menyimpan nilai pada node, serta `left_1016` dan `right_1016` yang berfungsi sebagai pointer ke anak kiri dan anak kanan.

```
package pekan9_2511531016;

public class Node_2511531016 {
    int data_1016;
    Node_2511531016 left_1016;
    Node_2511531016 right_1016;

    public Node_2511531016(int data_1016) {
        this.data_1016 = data_1016;
        left_1016 = null;
        right_1016 = null;
    }
}
```

Gambar 2.2 (Langkah 2 Node)

3. Kemudian, buat method `setLeft_1016()` dan `setRight_1016()` yang digunakan untuk menghubungkan node dengan anak kiri maupun anak kanan. Method ini hanya akan mengisi pointer apabila posisi anak masih kosong (null) sehingga hubungan antar node tetap terkontrol.

```
public void setLeft_1016(Node_2511531016 node_1016) {  
    if (left_1016 == null)  
        left_1016 = node_1016;  
}  
public void setRight_1016(Node_2511531016 node_1016) {  
    if (right_1016 == null)  
        right_1016 = node_1016;  
}
```

Gambar 2.3 (Langkah 3 Node)

4. Buat method `getLeft_1016()` dan `getRight_1016()`. Kedua method tersebut berfungsi untuk mengambil atau mengakses referensi anak kiri dan anak kanan dari suatu node sehingga dapat digunakan oleh class lain saat melakukan traversal atau pengolahan data pohon.

```
public Node_2511531016 getLeft_1016() {  
    return left_1016;  
}  
public Node_2511531016 getRight_1016() {  
    return right_1016;  
}
```

Gambar 2.4 (Langkah 4 Node)

5. Kemudian, buat method `getData_1016()` dan `setData_1016()`. Method `getData_1016()` digunakan untuk mengambil nilai data yang tersimpan pada node, sedangkan `setData_1016()` digunakan untuk mengubah nilai data pada node apabila diperlukan.

```
public int getData_1016() {  
    return data_1016;  
}  
public void setData_1016(int data_1016) {  
    this.data_1016 = data_1016;  
}
```

Gambar 2.5 (Langkah 5 Node)

6. Selanjutnya, buat method `printPreorder_1016()` untuk melakukan traversal Preorder. Pada metode ini node akar dikunjungi terlebih dahulu, kemudian lanjutkan ke subtree kiri dan subtree kanan secara rekursif. Hasil traversal akan ditampilkan ke layar sesuai urutan Preorder.

```
void printPreorder_1016(Node_2511531016 node_1016) {  
    if (node_1016 == null)  
        return;  
    System.out.print(node_1016.data_1016 + " ");  
    printPreorder_1016(node_1016.left_1016);  
    printPreorder_1016(node_1016.right_1016);  
}
```

Gambar 2.6 (Langkah 6 Node)

7. Buat method `printPostorder_1016()` yang berfungsi melakukan traversal Postorder. Proses traversal dimulai dari subtree kiri, kemudian subtree kanan, dan terakhir node akar. Metode ini menggunakan teknik rekursif untuk menelusuri seluruh node pada pohon.

```
void printPostorder_1016(Node_2511531016 node_1016) {  
    if (node_1016 == null)  
        return;  
    printPostorder_1016(node_1016.left_1016);  
    printPostorder_1016(node_1016.right_1016);  
    System.out.print(node_1016.data_1016 + " ");  
}
```

Gambar 2.7 (Langkah 7 Node)

8. Buat method `printInorder_1016()` yang digunakan untuk traversal Inorder. Traversal dilakukan dengan mengunjungi subtree kiri terlebih dahulu, kemudian node akar, dan terakhir subtree kanan. Metode ini sering digunakan untuk menampilkan data pada Binary Search Tree dalam keadaan terurut.

```
void printInorder_1016(Node_2511531016 node_1016) {  
    if (node_1016 == null)  
        return;  
    printInorder_1016(node_1016.left_1016);  
    System.out.print(node_1016.data_1016 + " ");  
    printInorder_1016(node_1016.right_1016);  
}
```

Gambar 2.8 (Langkah 8 Node)

9. Terakhir, buat method `print()` yang digunakan untuk menampilkan struktur Binary Tree dalam bentuk visual menyerupai pohon. Method ini memanfaatkan parameter `prefix_1016` dan `isTail_1016` untuk mengatur posisi cabang kiri dan kanan sehingga struktur pohon dapat terlihat lebih jelas saat ditampilkan pada console.

```
public String print() {
    return this.print("", true, "");
}
public String print(String prefix_1016, boolean isTail_1016, String
sb_1016) {
    if (right_1016 != null) {
        right_1016.print(prefix_1016 + (isTail_1016 ? "| " : " "), false,
sb_1016);
    }
    System.out.println(prefix_1016+(isTail_1016 ? "\\-- " : "/--
")+data_1016);
    if (left_1016 != null) {
        left_1016.print(prefix_1016+(isTail_1016 ? " " : "| "), true,
sb_1016);
    }
    return sb_1016;
}
```

Gambar 2.9 (Langkah 9 Node)

2.3.2 BTree_2511531016

1. Membuat kelas baru bernama *BTree_2511531016* dalam *package pekan9_2511531016*.

Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan9_2511531016
☐ Enclosing type:	
Name:	BTree_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.10 (Langkah 1 BTree)

2. Deklarasikan package dan class. Class ini berfungsi sebagai pengelola struktur Binary Tree. Di dalam class dideklarasikan variabel `root_1016` sebagai akar pohon dan `currentNode_1016` sebagai penunjuk node yang sedang aktif. Konstruktor `BTree_2511531016()` digunakan untuk menginisialisasi pohon dalam keadaan kosong dengan memberikan nilai null pada root.

```
package pekan9_2511531016;

public class BTree_2511531016 {
    private Node_2511531016 root_1016;
    private Node_2511531016 currentNode_1016;
    public BTree_2511531016() {
        root_1016 = null;
    }
}
```

Gambar 2.11 (Langkah 2 BTree)

3. Selanjutnya, buat method `search_1016()` yang berfungsi untuk mencari data pada Binary Tree. Pencarian dilakukan secara rekursif dengan menelusuri node saat ini, kemudian anak kiri, dan anak kanan. Jika data yang dicari ditemukan, method akan mengembalikan nilai true, sedangkan jika tidak ditemukan maka akan mengembalikan nilai false.

```
public boolean search_1016(int data_1016) {
    return search_1016(root_1016, data_1016);
}
private boolean search_1016(Node_2511531016 node_1016, int data_1016)
{
    if (node_1016.getData_1016() == data_1016)
        return true;
    if (node_1016.getLeft_1016() != null)
        if (search_1016(node_1016.getLeft_1016(), data_1016))
            return true;
    if (node_1016.getRight_1016() != null)
        if (search_1016(node_1016.getRight_1016(), data_1016))
            return true;
    return false;
}
```

Gambar 2.12 (Langkah 3 BTree)

4. Kemudian, buat method `printInorder_1016()`, `printPreorder_1016()`, dan `printPostorder_1016()`. Ketiga method ini digunakan untuk menampilkan isi Binary Tree menggunakan tiga teknik traversal yang berbeda. Setiap method akan memanggil method traversal yang telah dibuat pada class `Node_2511531016`.

```
public void printInorder_1016() {  
    root_1016.printInorder_1016(root_1016);  
}  
public void printPreorder_1016() {  
    root_1016.printPreorder_1016(root_1016);  
}  
public void printPostorder_1016() {  
    root_1016.printPostorder_1016(root_1016);  
}
```

Gambar 2.13 (Langkah 4 BTree)

5. Kemudian, buat method `getRoot_1016()` dan `isEmpty()`. Method `getRoot_1016()` digunakan untuk mengambil node akar dari Binary Tree, sedangkan method `isEmpty()` digunakan untuk memeriksa apakah pohon masih kosong dengan melihat nilai `root`.

```
public Node_2511531016 getRoot_1016() {  
    return root_1016;  
}  
public boolean isEmpty() {  
    return root_1016 == null;  
}
```

Gambar 2.14 (Langkah 5 BTree)

6. Buat method `countNodes_1016()` yang berfungsi untuk menghitung jumlah seluruh node yang terdapat pada Binary Tree. Method ini memanggil method rekursif lain yang bertugas melakukan perhitungan dari `root` hingga ke seluruh cabang pohon.

```
public int countNodes_1016() {  
    return countNodes_1016(root_1016);  
}
```

Gambar 2.15 (Langkah 6 BTree)

7. Kemudian, buat method rekursif `countNodes_1016(Node_2511531016 node_1016)` untuk menghitung jumlah node pada setiap subtree. Jika node bernilai null, method akan mengembalikan nilai 0. Jika node ada, maka program akan menghitung node saat ini ditambah jumlah node pada subtree kiri dan subtree kanan.

```
private int countNodes_1016(Node_2511531016 node_1016) {  
    int count_1016 = 1;  
    if (node_1016 == null) {  
        return 0;  
    } else {  
        count_1016 += countNodes_1016(node_1016.getLeft_1016());  
        count_1016 += countNodes_1016(node_1016.getRight_1016());  
        return count_1016;  
    }  
}
```

Gambar 2.16 (Langkah 7 BTree)

8. Buat method `print_1016()` dan `getCurrent_1016()`. Method `print_1016()` digunakan untuk menampilkan struktur Binary Tree secara visual dengan memanggil method `print()` pada node root. Sementara itu, method `getCurrent_1016()` berfungsi untuk mengambil atau mengakses node yang sedang aktif (current node) pada Binary Tree.

```
public void print_1016() {  
    root_1016.print();  
}  
public Node_2511531016 getCurrent_1016() {  
    return currentNode_1016;  
}
```

Gambar 2.17 (Langkah 8 BTree)

9. Terakhir buat method `setCurrent_1016()` dan `setRoot_1016()`. Method `setCurrent_1016()` digunakan untuk mengubah atau menentukan node yang sedang aktif pada Binary Tree. Sedangkan method `setRoot_1016()` digunakan untuk menetapkan atau mengganti node yang menjadi akar (root) dari pohon sehingga dapat digunakan sebagai titik awal berbagai operasi pada Binary Tree.

```

public void setCurrent_1016(Node_2511531016 node_1016) {
    this.currentNode_1016 = node_1016;
}
public void setRoot_1016(Node_2511531016 root_1016) {
    this.root_1016 = root_1016;
}

```

Gambar 2.18 (Langkah 9 BTree)

2.3.3 BTreeDriver_2511531016

1. Membuat kelas baru bernama *BTreeDriver_2511531016* dalam *package pekan9_2511531016*.

Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan9_2511531016
☐ Enclosing type:	
Name:	BTreeDriver_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.19 (Langkah 1 BTreeDriver)

2. Program dimulai dengan deklarasi package dan pembuatan class *BTreeDriver_2511531016* yang berisi method *main()* sebagai titik awal eksekusi program. Selanjutnya buat objek *tree_1016* dari class *BTree_2511531016* untuk merepresentasikan sebuah Binary Tree. Kemudian tampilkan jumlah simpul awal pada pohon yang masih kosong menggunakan method *countNodes_1016()*.

```

package pekan9_2511531016;

public class BTreeDriver_2511531016 {
    public static void main(String[] args) {
        // Membuat pohon
        BTree_2511531016 tree_1016 = new BTree_2511531016();
        System.out.print("Jumlah Simpul awal pohon: ");
    }
}

```

```
System.out.println(tree_1016.countNodes_1016());
```

Gambar 2.20 (Langkah 2 BTreeDriver)

3. Selanjutnya, buat node pertama dengan nilai data 1 yang akan dijadikan sebagai root dari Binary Tree. Setelah itu node tersebut diatur sebagai akar pohon menggunakan method `setRoot_1016()`. Program kemudian menampilkan jumlah simpul setelah root berhasil ditambahkan.

```
// Menambahkan simpul data 1
Node_2511531016 root_1016 = new Node_2511531016(1);
// Menjadikan simpul 1 sebagai root
tree_1016.setRoot_1016(root_1016);
System.out.println("Jumlah simpul jika hanya ada root: ");
System.out.println(tree_1016.countNodes_1016());
```

Gambar 2.21 (Langkah 3 BTreeDriver)

4. Buat beberapa objek node yang berisi data 2 hingga 9. Node-node tersebut nantinya akan digunakan untuk membentuk struktur Binary Tree yang lebih kompleks sehingga dapat dilakukan berbagai operasi traversal dan perhitungan jumlah simpul.

```
Node_2511531016 node2 = new Node_2511531016(2);
Node_2511531016 node3 = new Node_2511531016(3);
Node_2511531016 node4 = new Node_2511531016(4);
Node_2511531016 node5 = new Node_2511531016(5);
Node_2511531016 node6 = new Node_2511531016(6);
Node_2511531016 node7 = new Node_2511531016(7);
Node_2511531016 node8 = new Node_2511531016(8);
Node_2511531016 node9 = new Node_2511531016(9);
```

Gambar 2.22 (Langkah 4 BTreeDriver)

5. Kemudian seluruh node dihubungkan menggunakan method `setLeft_1016()` dan `setRight_1016()`. Hubungan ini membentuk struktur Binary Tree dengan root bernilai 1, yang memiliki cabang kiri dan kanan serta beberapa subtree di bawahnya.

```

root_1016.setLeft_1016(node2);
node2.setLeft_1016(node4);
node2.setRight_1016(node5);
node4.setRight_1016(node8);
root_1016.setRight_1016(node3);
node3.setLeft_1016(node6);
node3.setRight_1016(node7);
node6.setLeft_1016(node9);

```

Gambar 2.23 (Langkah 5 BTreeDriver)

6. Setelah struktur pohon selesai dibuat, program mengatur root sebagai current node menggunakan method `setCurrent_1016()`. Kemudian program menampilkan data pada current node dan menghitung jumlah seluruh simpul yang terdapat pada Binary Tree menggunakan method `countNodes_1016()`.

```

// Set root
tree_1016.setCurrent_1016(tree_1016.getRoot_1016());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree_1016.getCurrent_1016().getData_1016());
System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
System.out.println(tree_1016.countNodes_1016());

```

Gambar 2.24 (Langkah 6 BTreeDriver)

7. Terakhir program menampilkan isi Binary Tree menggunakan tiga metode traversal yaitu Inorder, Preorder, dan Postorder. Selain itu, program juga menampilkan struktur Binary Tree dalam bentuk pohon menggunakan method `print_1016()` sehingga hubungan antar node dapat terlihat dengan lebih jelas.

```

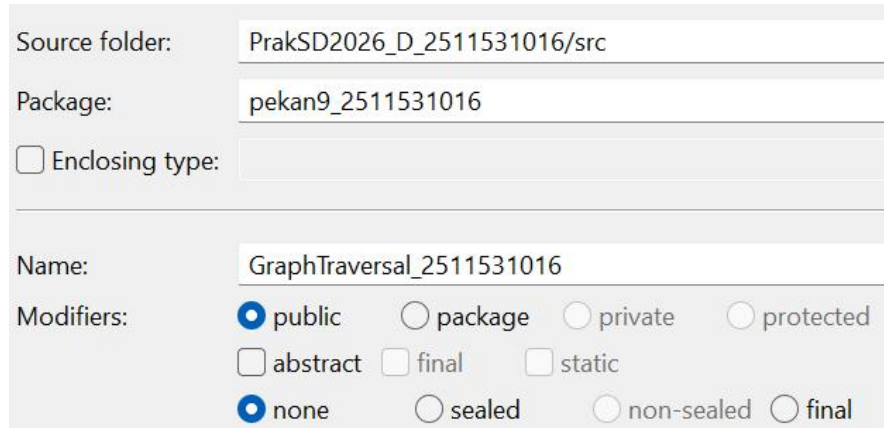
System.out.println("InOrder: ");
tree_1016.printInorder_1016();
System.out.println("\nPreorder: ");
tree_1016.printPreorder_1016();
System.out.println("\nPostorder: ");
tree_1016.printPostorder_1016();
System.out.println("\nMenampilkan simpul dalam bentuk pohon");
tree_1016.print_1016();

```

Gambar 2.25 (Langkah 7 BTreeDriver)

2.3.4 GraphTraversal_2511531016

1. Membuat kelas baru bernama *GraphTraversal_2511531016* dalam *package pekan9_2511531016*.



Source folder: PrakSD2026_D_2511531016/src

Package: pekan9_2511531016

☐ Enclosing type:

Name: GraphTraversal_2511531016

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.26 (Langkah 1 GraphTraversal)

2. Program dimulai dengan deklarasi package, import library `java.util.*`, serta pembuatan class `GraphTraversal_2511531016`. Pada bagian ini juga dideklarasikan variabel `graph_1016` bertipe `Map<String, List<String>>` yang digunakan untuk menyimpan data graf dalam bentuk *adjacency list*. Struktur ini memudahkan program dalam menyimpan hubungan antar simpul.

```
package pekan9_2511531016;  
import java.util.*;  
  
public class GraphTraversal_2511531016 {  
    private Map<String, List<String>> graph_1016 = new HashMap<>();  
}
```

Gambar 2.27 (Langkah 2 GraphTraversal)

3. Buat method `addEdge()` yang berfungsi untuk menambahkan hubungan antar simpul pada graf. Karena graf yang digunakan bersifat tidak berarah (*undirected graph*), maka setiap simpul akan saling ditambahkan ke daftar tetangga masing-masing.

```
// Menambahkan edge (graf tak berarah)
public void addEdge(String node1, String node2) {
    graph_1016.putIfAbsent(node1, new ArrayList<>());
    graph_1016.putIfAbsent(node2, new ArrayList<>());
    graph_1016.get(node1).add(node2);
    graph_1016.get(node2).add(node1);
}
```

Gambar 2.28 (Langkah 3 GraphTraversal)

4. Kemudian, buat method `printGraph()` yang digunakan untuk menampilkan isi graf dalam bentuk *adjacency list*. Program akan membaca setiap simpul yang ada di dalam graf dan menampilkan daftar simpul lain yang terhubung dengannya.

```
// Menampilkan graf awal
public void printGraph() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node_1016 : graph_1016.keySet()) {
        System.out.print(node_1016 + " -> ");
        List<String> neighbors_1016 = graph_1016.get(node_1016);
        System.out.println(String.join(", ", neighbors_1016));
    }
    System.out.println();
}
```

Gambar 2.29 (Langkah 4 GraphTraversal)

5. Buat method `dfs()` untuk melakukan penelusuran graf menggunakan algoritma Depth First Search (DFS). Method ini membuat variabel `visited_1016` untuk mencatat simpul yang sudah dikunjungi sehingga tidak terjadi pengulangan saat proses penelusuran berlangsung.

```
// DFS rekursif
public void dfs(String start_1016) {
    Set<String> visited_1016 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper(start_1016, visited_1016);
    System.out.println();
}
```

Gambar 2.30 (Langkah 5 GraphTraversal)

6. Buat method dfsHelper() yang bekerja secara rekursif untuk menjalankan proses DFS. Program akan memeriksa apakah suatu simpul sudah pernah dikunjungi, kemudian menampilkan simpul tersebut dan melanjutkan penelusuran ke seluruh tetangganya secara mendalam.

```
private void dfsHelper(String current_1016, Set<String> visited_1016)
{
    if (visited_1016.contains(current_1016)) return;
    visited_1016.add(current_1016);
    System.out.print(current_1016 + " ");
    for (String neighbor_1016 : graph_1016.getDefault(current_1016,
        new ArrayList<>())) {
        dfsHelper(neighbor_1016, visited_1016);
    }
}
```

Gambar 2.31 (Langkah 6 GraphTraversal)

7. Buat method bfs() yang digunakan untuk melakukan penelusuran menggunakan algoritma Breadth First Search (BFS). Pada method ini digunakan Queue untuk menyimpan urutan simpul yang akan dikunjungi dan Set untuk menyimpan simpul yang sudah diproses.

```
// BFS iteratif
public void bfs(String start_1016) {
    Set<String> visited_1016 = new HashSet<>();
    Queue<String> queue_1016 = new LinkedList<>();
    queue_1016.add(start_1016);
    visited_1016.add(start_1016);
    System.out.println("Penelusuran BFS:");
}
```

Gambar 2.32 (Langkah 7 GraphTraversal)

8. Buat perulangan while yang akan berjalan selama antrian masih berisi data. Program akan mengambil simpul yang berada di bagian depan antrian, menampilkannya ke layar, kemudian memeriksa seluruh tetangga dari simpul tersebut.

```
while (!queue_1016.isEmpty()) {
    String current_1016 = queue_1016.poll();
    System.out.print(current_1016 + " ");
}
```

```
for (String neighbor_1016 : graph_1016.getDefault(current_1016,
new ArrayList<>())) {
```

Gambar 2.33 (Langkah 8 GraphTraversal)

9. Kemudian, program melakukan pengecekan apakah simpul tetangga sudah pernah dikunjungi atau belum. Jika belum, simpul tersebut akan dimasukkan ke dalam antrian dan ditandai sebagai sudah dikunjungi sehingga tidak diproses kembali pada iterasi berikutnya.

```
if (!visited_1016.contains(neighbor_1016)) {
queue_1016.add(neighbor_1016);
visited_1016.add(neighbor_1016);
}
}
}
System.out.println();
```

Gambar 2.34 (Langkah 9 GraphTraversal)

10. Buat method main() sebagai titik awal eksekusi program. Pada bagian ini dibuat objek graph_1016 dari class GraphTraversal_2511531016, kemudian tambahkan beberapa hubungan antar simpul menggunakan method addEdge() untuk membentuk sebuah graf sederhana.

```
// Main
public static void main(String[] args) {
GraphTraversal_2511531016 graph_1016 = new
GraphTraversal_2511531016();
// Contoh graf: A-B, A-C, B-D, B-E
graph_1016.addEdge("A", "B");
graph_1016.addEdge("A", "C");
graph_1016.addEdge("B", "D");
graph_1016.addEdge("B", "E");
```

Gambar 2.35 (Langkah 10 GraphTraversal)

11. Terakhir, program menampilkan graf yang telah dibuat menggunakan method printGraph(). Setelah itu dilakukan proses traversal menggunakan

algoritma DFS dan BFS yang dimulai dari simpul A sehingga urutan kunjungan simpul dapat ditampilkan pada output program.

```
// Cetak graf awal
System.out.println("Graf Awal adalah: ");
graph_1016.printGraph();
// Lakukan penelusuran
graph_1016.dfs("A");
graph_1016.bfs("A");
```

Gambar 2.36 (Langkah 11 GraphTraversal)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, program Binary Tree berhasil dibuat dan dijalankan sesuai dengan rancangan yang telah ditentukan. Hal ini dapat dilihat dari jumlah simpul yang ditampilkan oleh program, mulai dari kondisi awal pohon yang kosong hingga setelah seluruh node ditambahkan. Program juga mampu membentuk hubungan antar node dengan benar sehingga struktur pohon yang dihasilkan sesuai dengan desain yang telah dibuat sebelumnya. Keberhasilan ini menunjukkan bahwa proses pembuatan node, penentuan root, serta penghubungan antara anak kiri dan anak kanan telah berjalan dengan baik.

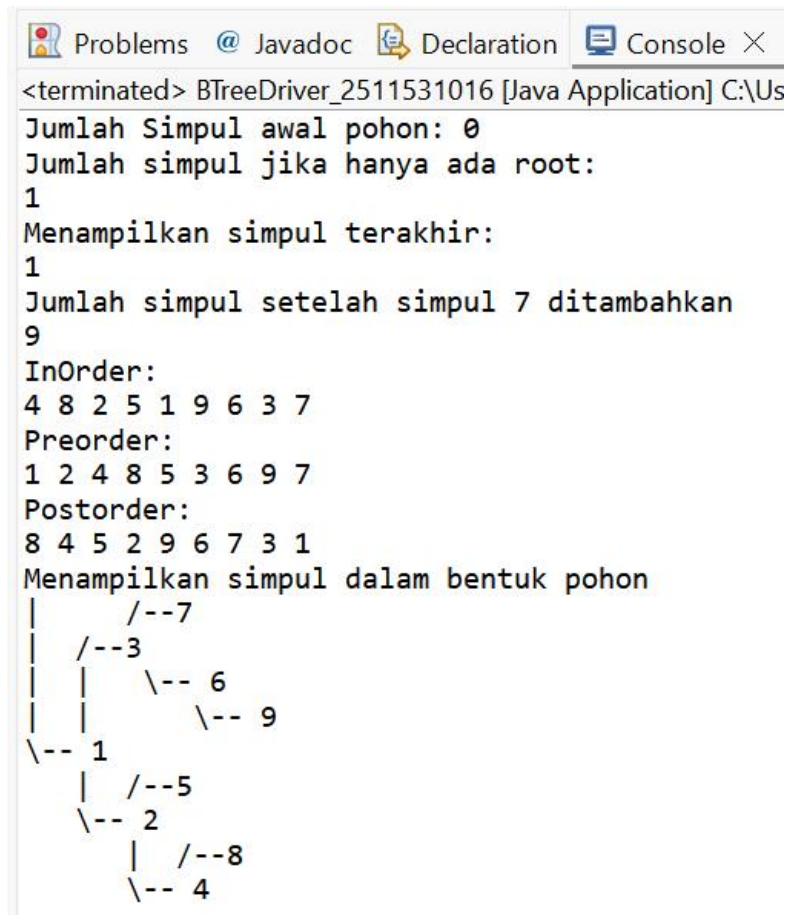
Selain itu, hasil traversal InOrder, PreOrder, dan PostOrder menunjukkan bahwa setiap metode menghasilkan urutan kunjungan node yang berbeda sesuai dengan karakteristik masing-masing. Hasil tersebut membuktikan bahwa algoritma traversal telah diimplementasikan dengan benar pada Binary Tree. Tampilan struktur pohon dalam bentuk visual juga membantu dalam memahami posisi setiap node dan hubungan antar cabang pada pohon. Dari praktikum ini dapat dipahami bahwa cara penyusunan node akan memengaruhi hasil traversal yang diperoleh, sehingga struktur pohon menjadi faktor penting dalam proses pengolahan data menggunakan Binary Tree.

BAB III

KESIMPULAN

3.1 Hasil Praktikum

3.1.1 BTreeDriver_2511531016



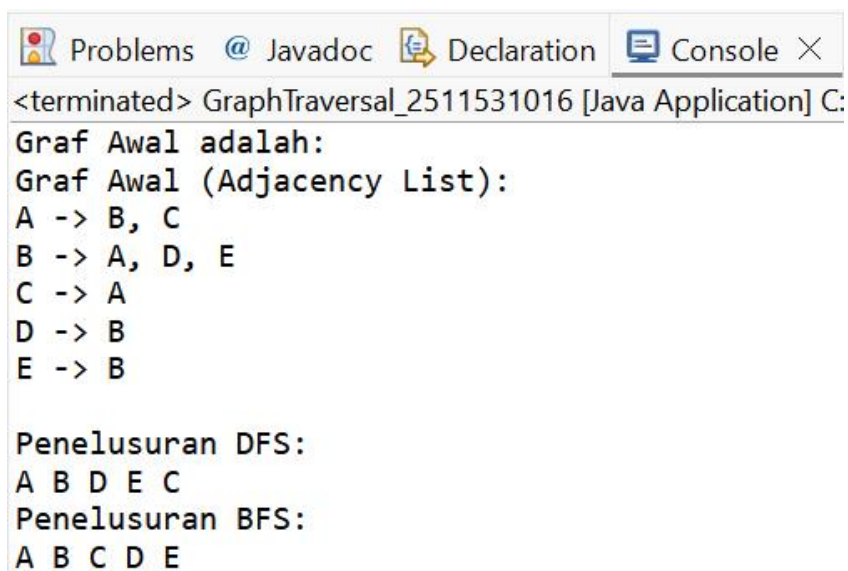
```
Problems @ Javadoc Declaration Console X
<terminated> BTreeDriver_2511531016 [Java Application] C:\Us
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root:
1
Menampilkan simpul terakhir:
1
Jumlah simpul setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /--7
|      /--3
|      |      \-- 6
|      |      \-- 9
|-- 1
|      /--5
|-- 2
|      /--8
|      \-- 4
```

Gambar 3.1 (Hasil Praktikum BTreeDriver)

Output program menunjukkan bahwa Binary Tree berhasil dibuat dan diisi dengan 9 simpul, dimulai dari node 1 sebagai root hingga node 9 sebagai anak pada beberapa cabang pohon. Program kemudian menampilkan hasil traversal InOrder, PreOrder, dan PostOrder yang masing-masing menghasilkan urutan kunjungan node yang berbeda sesuai aturan traversalnya.

Selain itu, program juga menampilkan struktur Binary Tree dalam bentuk visual sehingga hubungan antara root, anak kiri, dan anak kanan dapat terlihat dengan jelas. Dari hasil tersebut dapat disimpulkan bahwa proses pembentukan pohon, perhitungan jumlah simpul, serta traversal pada Binary Tree telah berjalan dengan benar sesuai dengan struktur yang dibuat pada program.

3.1.2 GraphTraversal_2511531016



```
<terminated> GraphTraversal_2511531016 [Java Application] C:
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 3.2 (Hasil Praktikum GraphTraversal)

Output program menampilkan representasi graf dalam bentuk *adjacency list*, di mana simpul A terhubung dengan B dan C, simpul B terhubung dengan A, D, dan E, sedangkan simpul C, D, dan E masing-masing terhubung kembali ke simpul yang terhubung dengannya. Setelah graf berhasil dibuat, program melakukan penelusuran menggunakan algoritma DFS dan BFS yang dimulai dari simpul A. Hasil DFS menghasilkan urutan A B D E C karena penelusuran dilakukan sedalam mungkin pada satu cabang sebelum berpindah ke cabang lain.

Sementara itu, hasil BFS menghasilkan urutan A B C D E karena penelusuran dilakukan per level, yaitu mengunjungi seluruh tetangga terdekat terlebih dahulu sebelum melanjutkan ke level berikutnya. Hasil tersebut menunjukkan bahwa kedua algoritma traversal graf telah berjalan dengan benar sesuai karakteristik masing-masing.

3.2 Saran Pengembangan

Program yang telah dibuat masih menggunakan data node yang ditambahkan secara manual di dalam kode. Pada pengembangan berikutnya, program dapat dibuat lebih fleksibel dengan menambahkan fitur input data melalui keyboard sehingga pengguna dapat menentukan sendiri jumlah dan nilai node yang ingin dimasukkan. Dengan demikian, program tidak hanya dapat digunakan pada satu contoh kasus, tetapi juga dapat diuji pada berbagai bentuk dan ukuran Binary Tree yang berbeda.

Selain itu, program dapat dikembangkan dengan menambahkan fitur pencarian, penyisipan, dan penghapusan node secara dinamis. Pengembangan ke arah Binary Search Tree (BST) juga dapat menjadi langkah selanjutnya karena memungkinkan proses pencarian data dilakukan dengan lebih efisien. Dengan adanya fitur-fitur tersebut, program tidak hanya berfungsi sebagai media pembelajaran traversal pohon, tetapi juga dapat memberikan gambaran yang lebih lengkap mengenai penerapan struktur data tree dalam pengembangan perangkat lunak.

DAFTAR PUSTAKA

- [1] I. Sahyuni, A. C. Bella, M. Rijal, S. Ramadhani, E. B. Upa', S. D. Azzahra, dan K. R. Manullang, "Penting Struktur Data Tree dalam Mengelolah Data Komputer," *Jejak Digital: Jurnal Ilmiah Multidisiplin*.
- [2] A. Agustian, E. Rosa, R. Destian, dan L. S. Riza, "Meningkatkan Berpikir Komputasi Siswa Melalui Aktivitas Unplugged: Studi Action Research pada Binary Search Trees," *Jurnal Sains dan Ilmu Terapan*.

TUGAS PRAKTIKUM PEKAN 9

Pencarian Jalur pada suatu Peta Lokasi

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (*Breadth First Search*) dan DFS (*Depth First Search*) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

2. Kode Program

2.1 PetaBandara_2511531016

```
package pekan9_2511531016;

import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;
import javax.swing.*;

public class PetaBandara_2511531016 extends JFrame {
    private Map<String, List<String>> graph_1016 = new HashMap<>();
    private JComboBox<String> cmbAwal_1016;
    private JComboBox<String> cmbTujuan_1016;
    private JTextArea hasil_1016;
    private JLabel jalur_1016;
    private JLabel nodeDikunjungi_1016;
    private JLabel jumlahNode_1016;
    private GraphPanel_1016 panelGraf_1016;
    private List<String> visitedNodes_1016 = new ArrayList<>();

    public PetaBandara_2511531016() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setSize(1050, 750);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        createGraph_1016();
        initComponents_1016();
    }

    private void initComponents_1016() {
        JPanel top_1016 = new JPanel();
        cmbAwal_1016 = new JComboBox<>(graph_1016.keySet().toArray(new String[0]));
        cmbTujuan_1016 = new JComboBox<>(graph_1016.keySet().toArray(new String[0]));
    }
}
```

```

JButton btnBFS_1016 = new JButton("BFS");
JButton btnDFS_1016 = new JButton("DFS");
JButton btnReset_1016 = new JButton("RESET");

top_1016.add(new JLabel("Lokasi Awal :"));
top_1016.add(cmbAwal_1016);
top_1016.add(new JLabel("Lokasi Tujuan :"));
top_1016.add(cmbTujuan_1016);
top_1016.add(btnBFS_1016);
top_1016.add(btnDFS_1016);
top_1016.add(btnReset_1016);
add(top_1016, BorderLayout.NORTH);

panelGraf_1016 = new GraphPanel_1016();
add(panelGraf_1016, BorderLayout.CENTER);
JPanel bawah_1016 = new JPanel();
bawah_1016.setLayout(new GridLayout(4, 1));
hasil_1016 = new JTextArea();
hasil_1016.setEditable(false);
jalur_1016 = new JLabel("Jalur :");
nodeDikunjungi_1016 = new JLabel("Node Dikunjungi :");
jumlahNode_1016 = new JLabel("Jumlah Node Dikunjungi : 0");

bawah_1016.add(new JScrollPane(hasil_1016));
bawah_1016.add(jalur_1016);
bawah_1016.add(nodeDikunjungi_1016);
bawah_1016.add(jumlahNode_1016);
add(bawah_1016, BorderLayout.SOUTH);
btnBFS_1016.addActionListener(e -> bfs_1016());
btnDFS_1016.addActionListener(e -> dfs_1016());
btnReset_1016.addActionListener(e -> resetGraph_1016());
}

private void addEdge_1016(String a, String b) {
    graph_1016.putIfAbsent(a, new ArrayList<>());
    graph_1016.putIfAbsent(b, new ArrayList<>());
    graph_1016.get(a).add(b);
    graph_1016.get(b).add(a);
}

private void createGraph_1016() {
    addEdge_1016("Terminal 1", "Terminal 2");
    addEdge_1016("Terminal 2", "Terminal 3");

    addEdge_1016("Terminal 1", "Parkir");
    addEdge_1016("Terminal 2", "Area Check-In");
    addEdge_1016("Terminal 3", "Area Bagasi");

    addEdge_1016("Parkir", "Area Check-In");
    addEdge_1016("Area Check-In", "Area Bagasi");

    addEdge_1016("Area Check-In", "Security Check");
    addEdge_1016("Security Check", "Ruang Tunggu");

    addEdge_1016("Ruang Tunggu", "Gate A");
    addEdge_1016("Ruang Tunggu", "Gate B");

    addEdge_1016("Gate A", "Terminal 1");
    addEdge_1016("Gate B", "Terminal 2");

    addEdge_1016("Area Bagasi", "Terminal 3");
    addEdge_1016("Area Bagasi", "Parkir");
}

private void bfs_1016() {
    String start_1016 = cmbAwal_1016.getSelectedItem().toString();
    String goal_1016 = cmbTujuan_1016.getSelectedItem().toString();
    Queue<String> queue_1016 = new LinkedList<>();
    Set<String> visited_1016 = new HashSet<>();
    Map<String, String> parent_1016 = new HashMap<>();
    visitedNodes_1016.clear();
    queue_1016.add(start_1016);
    visited_1016.add(start_1016);
    while (!queue_1016.isEmpty()) {
        String current_1016 = queue_1016.poll();
        visitedNodes_1016.add(current_1016);
        if (current_1016.equals(goal_1016))
            break;
        for (String neighbor_1016 : graph_1016.get(current_1016)) {
            if (!visited_1016.contains(neighbor_1016)) {
                visited_1016.add(neighbor_1016);
                parent_1016.put(neighbor_1016, current_1016);
                queue_1016.add(neighbor_1016);
            }
        }
    }
    tampilkanHasil_1016(parent_1016, start_1016, goal_1016, "BFS");
}

```

```

private void dfs_1016() {
    String start_1016 = cmbAwal_1016.getSelectedItem().toString();
    String goal_1016 = cmbTujuan_1016.getSelectedItem().toString();
    Stack<String> stack_1016 = new Stack<>();
    Set<String> visited_1016 = new HashSet<>();
    Map<String, String> parent_1016 = new HashMap<>();
    visitedNodes_1016.clear();
    stack_1016.push(start_1016);
    while (!stack_1016.isEmpty()) {
        String current_1016 = stack_1016.pop();
        if (visited_1016.contains(current_1016))
            continue;
        visited_1016.add(current_1016);
        visitedNodes_1016.add(current_1016);
        if (current_1016.equals(goal_1016))
            break;
        for (String neighbor_1016 : graph_1016.get(current_1016)) {
            if (!visited_1016.contains(neighbor_1016)) {
                parent_1016.put(neighbor_1016, current_1016);
                stack_1016.push(neighbor_1016);
            }
        }
    }
    tampilkanHasil_1016(parent_1016, start_1016, goal_1016, "DFS");
}

private void tampilkanHasil_1016(Map<String, String> parent_1016, String start_1016, String goal_1016, String metode_1016) {
    List<String> path_1016 = new ArrayList<>();
    String current_1016 = goal_1016;
    while (current_1016 != null) {
        path_1016.add(current_1016);
        current_1016 = parent_1016.get(current_1016);
    }

    Collections.reverse(path_1016);
    hasil_1016.setText("Metode : " + metode_1016);
    displayPath_1016(path_1016);
    nodeDikunjungi_1016.setText("Node Dikunjungi : " + visitedNodes_1016);
    jumlahNode_1016.setText("Jumlah Node Dikunjungi : " + visitedNodes_1016.size());
    displayGraph_1016();
}

private void resetGraph_1016() {
    visitedNodes_1016.clear();
    hasil_1016.setText("");
    jalur_1016.setText("Jalur :");
    nodeDikunjungi_1016.setText("Node Dikunjungi :");
    jumlahNode_1016.setText("Jumlah Node Dikunjungi : 0");
    displayGraph_1016();
}

private void displayGraph_1016() {
    panelGraf_1016.repaint();
}

private void displayPath_1016(List<String> path_1016) {
    jalur_1016.setText("Jalur : " + String.join(" -> ", path_1016));
}

class GraphPanel_1016 extends JPanel {
    Map<String, Point> posisi_1016 = new HashMap<>();
    public GraphPanel_1016() {
        posisi_1016.put("Terminal 1", new Point(150, 80));
        posisi_1016.put("Terminal 2", new Point(350, 80));
        posisi_1016.put("Terminal 3", new Point(550, 80));

        posisi_1016.put("Parkir", new Point(80, 220));
        posisi_1016.put("Area Check-In", new Point(350, 220));
        posisi_1016.put("Area Bagasi", new Point(620, 220));

        posisi_1016.put("Security Check", new Point(350, 320));

        posisi_1016.put("Ruang Tunggu", new Point(350, 420));

        posisi_1016.put("Gate A", new Point(220, 520));
        posisi_1016.put("Gate B", new Point(480, 520));
    }

    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        for (String node_1016 : graph_1016.keySet()) {
            Point p1 = posisi_1016.get(node_1016);
            for (String tetangga_1016 : graph_1016.get(node_1016)) {
                Point p2 = posisi_1016.get(tetangga_1016);
                g.drawLine(p1.x, p1.y, p2.x, p2.y);
            }
        }
        for (String node_1016 : posisi_1016.keySet()) {
            Point p = posisi_1016.get(node_1016);
            if (visitedNodes_1016.contains(node_1016))
                g.setColor(Color.GREEN);
            else
                g.setColor(Color.LIGHT_GRAY);
            g.fillOval(p.x - 25, p.y - 25, 50, 50);
            g.setColor(Color.BLACK);
            g.drawOval(p.x - 25, p.y - 25, 50, 50);
            g.drawString(node_1016, p.x - 30, p.y - 35);
        }
    }
}

```

```

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new PetaBandara_2511531016().setVisible(true);
        });
    }
}

```

Penjelasan :

Program GUI ini dibuat untuk mengimplementasikan pencarian jalur pada graf menggunakan algoritma BFS dan DFS. Pada awal program, dibuat struktur graf menggunakan HashMap yang menyimpan hubungan antar node berupa lokasi-lokasi di area bandara. Selanjutnya antarmuka dibangun menggunakan komponen Java Swing seperti JFrame, JPanel, JComboBox, JButton, JTextArea, dan JLabel.

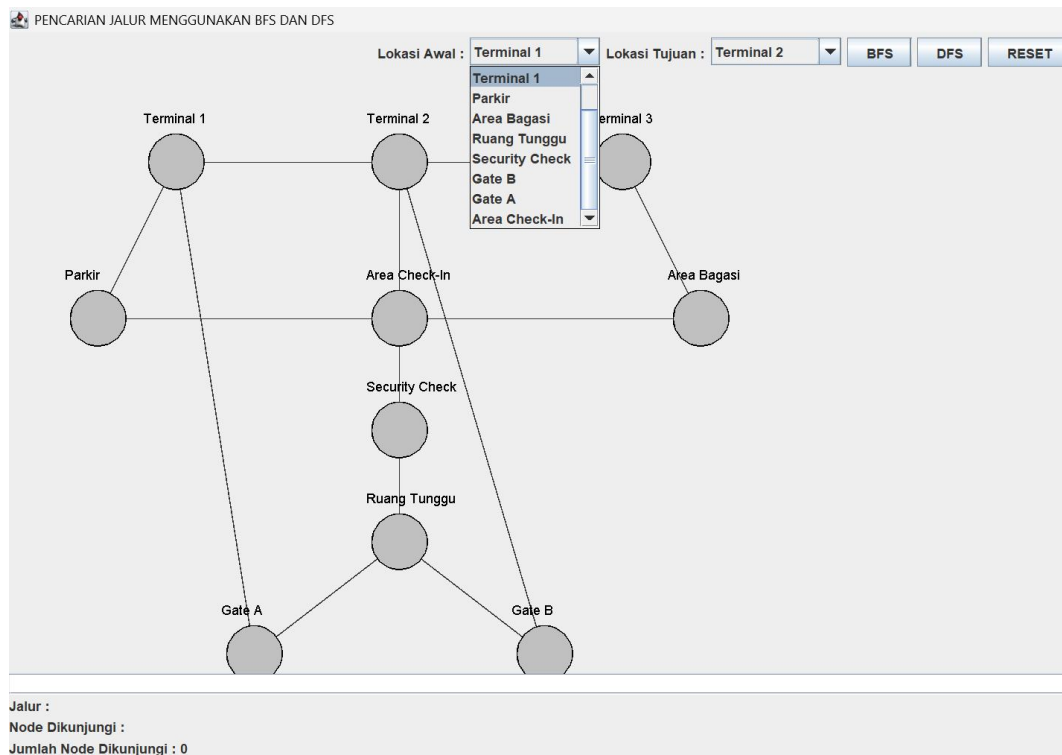
Pengguna dapat memilih lokasi awal dan lokasi tujuan melalui ComboBox yang tersedia, kemudian menjalankan pencarian menggunakan tombol BFS atau DFS. Program juga menyediakan tombol Reset untuk menghapus hasil pencarian dan mengembalikan tampilan graf ke kondisi awal. Selain itu, method seperti addEdge() digunakan untuk menambahkan hubungan antar node, sedangkan createGraph() berfungsi membentuk keseluruhan peta yang akan digunakan dalam proses pencarian.

Pada proses pencarian, method bfs() memanfaatkan struktur data Queue untuk menelusuri node secara melebar, sedangkan method dfs() menggunakan Stack untuk menelusuri node hingga ke kedalaman tertentu terlebih dahulu. Setiap node yang dikunjungi akan disimpan ke dalam daftar visitedNodes_1016, kemudian ditampilkan pada area hasil beserta jumlah node yang dieksplorasi. Method displayPath() digunakan untuk menampilkan jalur yang ditemukan dari lokasi awal ke lokasi tujuan, sedangkan displayGraph() berfungsi memperbarui tampilan visual graf.

Pada bagian GraphPanel, method paintComponent() digunakan untuk menggambar node dan edge serta memberikan warna berbeda pada node yang telah dikunjungi sehingga proses penelusuran dapat diamati secara langsung. Dengan adanya visualisasi tersebut, pengguna dapat lebih mudah

memahami perbedaan cara kerja BFS dan DFS dalam menemukan jalur pada sebuah graf.

3. Output Kode Program



3.1 Deskripsi Peta

Pada program ini, bandara direpresentasikan dalam bentuk graf, yaitu kumpulan simpul (node) dan sisi (edge) yang saling terhubung. Setiap node menggambarkan lokasi penting yang ada di bandara, sedangkan edge menggambarkan jalur yang dapat dilalui untuk berpindah dari satu lokasi ke lokasi lainnya. Peta ini digunakan sebagai media untuk menerapkan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) dalam mencari jalur dari lokasi awal menuju lokasi tujuan.

Berdasarkan gambar, terdapat beberapa fasilitas yang menjadi node dalam graf, yaitu Terminal 1, Terminal 2, Terminal 3, Parkir, Area Check-In,

Security Check, Ruang Tunggu, Gate A, Gate B, dan Area Bagasi. Setiap lokasi memiliki hubungan dengan lokasi lain melalui jalur yang telah ditentukan. Misalnya, Terminal 1 terhubung dengan Terminal 2 dan Parkir, Area Check-In terhubung dengan Security Check, sedangkan Ruang Tunggu terhubung dengan Gate A dan Gate B. Dengan adanya hubungan tersebut, program dapat melakukan pencarian rute antar lokasi sesuai pilihan pengguna.

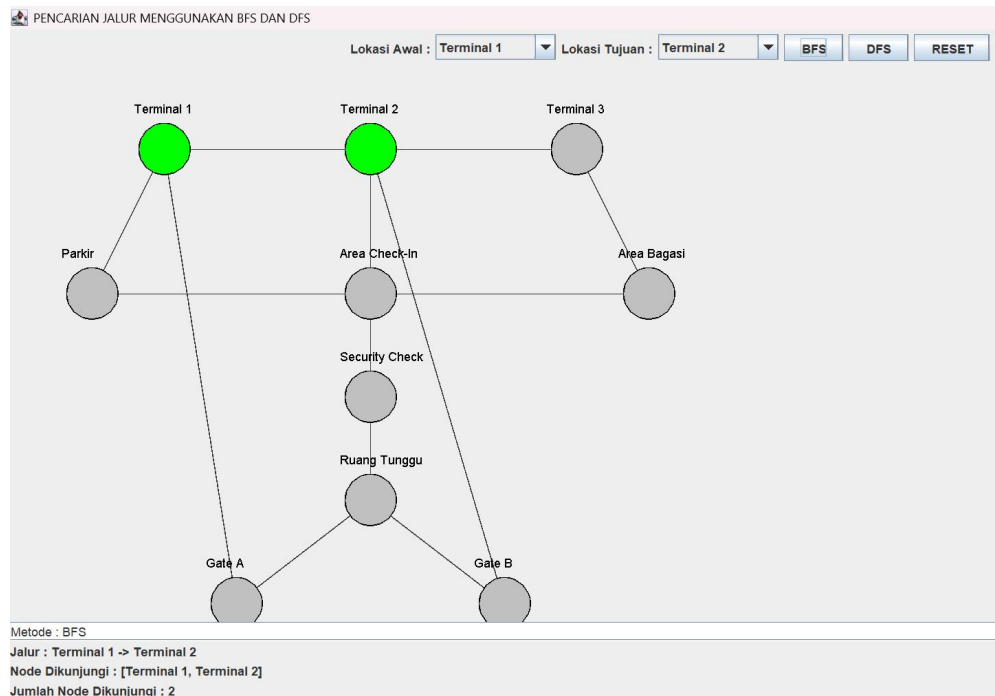
3.2 Jumlah Node dan Edge

Graf pada peta bandara ini terdiri dari **10 node** dan **15 edge** yang membentuk jaringan jalur antar fasilitas di dalam bandara. Setiap node memiliki peran sebagai lokasi yang dapat dikunjungi oleh pengguna, sedangkan edge berfungsi sebagai penghubung yang memungkinkan perpindahan dari satu lokasi ke lokasi lain. Struktur graf yang digunakan cukup sederhana namun sudah mampu menggambarkan hubungan antar area penting seperti terminal, area check-in, ruang tunggu, hingga gate keberangkatan.

Dengan adanya 15 jalur penghubung tersebut, algoritma BFS dan DFS dapat digunakan untuk mencari rute dari lokasi awal menuju lokasi tujuan. BFS akan menelusuri node berdasarkan tingkat kedekatan sehingga cenderung menemukan jalur terpendek terlebih dahulu, sedangkan DFS akan menelusuri satu jalur hingga mencapai titik terdalam sebelum kembali ke percabangan sebelumnya. Oleh karena itu, peta bandara ini dapat digunakan untuk membandingkan cara kerja kedua algoritma dalam proses pencarian jalur pada sebuah graf.

4. Hasil

4.1 BFS

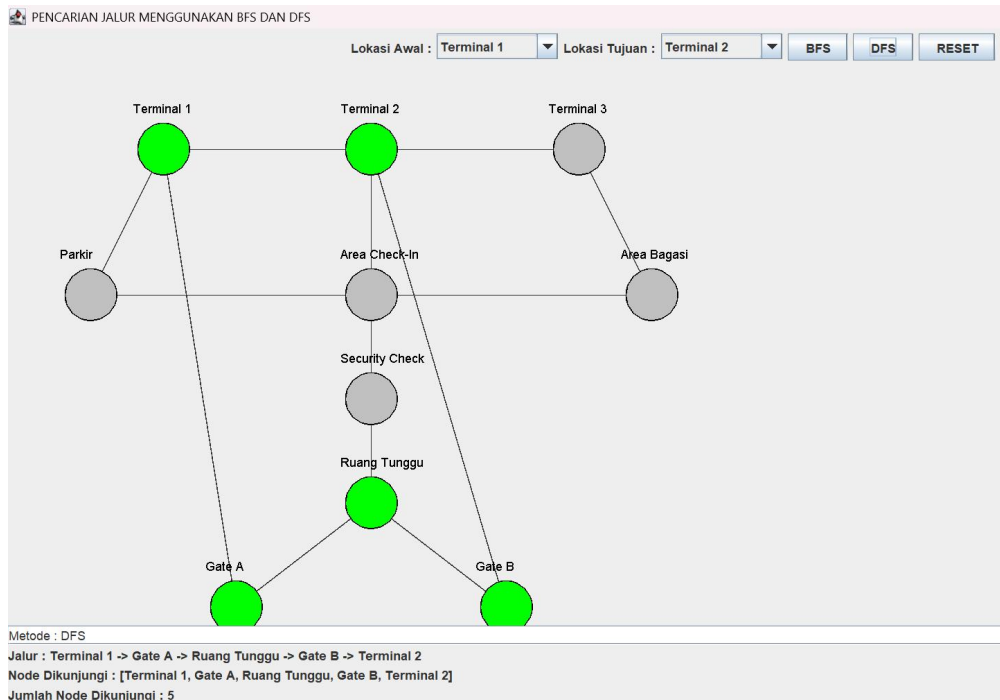


Penjelasan :

Pada pengujian menggunakan algoritma **Breadth First Search (BFS)**, lokasi awal dipilih **Terminal 1** dan lokasi tujuan **Terminal 2**. Hasil yang ditampilkan menunjukkan jalur **Terminal 1 → Terminal 2** dengan jumlah node yang dikunjungi sebanyak **2 node**, yaitu Terminal 1 dan Terminal 2. Node yang berwarna hijau menunjukkan node yang termasuk dalam jalur pencarian.

Hasil tersebut menunjukkan bahwa BFS berhasil menemukan jalur tujuan dengan cepat karena Terminal 2 merupakan tetangga langsung dari Terminal 1. Sesuai karakteristik BFS yang menelusuri node berdasarkan tingkat kedekatan, algoritma langsung menemukan tujuan tanpa perlu mengunjungi node lain yang lebih jauh. Oleh karena itu, jalur yang diperoleh merupakan jalur paling singkat dengan proses pencarian yang lebih efisien.

4.2 DFS



Penjelasan :

Pada pengujian menggunakan algoritma **Depth First Search (DFS)**, lokasi awal tetap **Terminal 1** dan lokasi tujuan **Terminal 2**. Hasil yang diperoleh adalah jalur **Terminal 1 → Gate A → Ruang Tunggu → Gate B → Terminal 2** dengan jumlah node yang dikunjungi sebanyak **5 node**. Node yang berwarna hijau menunjukkan node-node yang telah dilalui selama proses pencarian.

Berbeda dengan BFS, DFS tidak langsung memilih jalur terdekat menuju tujuan. Algoritma ini menelusuri satu cabang terlebih dahulu hingga mencapai titik terdalam sebelum kembali ke node sebelumnya. Karena itu, DFS melewati beberapa node seperti Gate A, Ruang Tunggu, dan Gate B sebelum akhirnya mencapai Terminal 2. Hasil ini menunjukkan bahwa DFS mampu menemukan tujuan, namun jalur yang dihasilkan tidak selalu merupakan jalur terpendek dibandingkan BFS.

5. Kesimpulan Perbandingan BFS dan DFS

Berdasarkan hasil pengujian pada peta bandara, algoritma BFS (Breadth First Search) dan DFS (Depth First Search) sama-sama berhasil menemukan jalur dari Terminal 1 menuju Terminal 2, namun menghasilkan proses pencarian yang berbeda. Pada BFS, jalur yang ditemukan adalah Terminal 1 → Terminal 2 dengan hanya mengunjungi 2 node. Hal ini terjadi karena BFS mencari node berdasarkan tingkat kedekatan sehingga dapat menemukan jalur tujuan yang paling dekat terlebih dahulu. Oleh karena itu, BFS menghasilkan jalur yang lebih pendek dan lebih efisien pada kasus ini.

Sementara itu, DFS menghasilkan jalur Terminal 1 → Gate A → Ruang Tunggu → Gate B → Terminal 2 dengan mengunjungi 5 node. Hal ini menunjukkan bahwa DFS menelusuri satu jalur secara mendalam terlebih dahulu sebelum mencoba jalur lainnya. Akibatnya, jumlah node yang dikunjungi lebih banyak dan jalur yang diperoleh tidak sependek BFS. Dari hasil pengujian tersebut dapat disimpulkan bahwa **BFS** lebih efektif untuk mencari jalur terpendek, sedangkan **DFS** lebih fokus pada eksplorasi jalur hingga kedalaman tertentu dan tidak menjamin menemukan jalur terpendek. Dengan demikian, untuk kasus pencarian rute pada peta bandara ini, BFS memberikan hasil yang lebih optimal dibandingkan DFS.